Universidade Federal de Sergipe — **PROCC**
Projeto e Análise de Algoritmos · Período 2026.2
Módulo II — Otimização, Grafos e Processamento de Cadeias

Atividade 2 (AV2) — Relatório Técnico

Recuperação e seleção otimizada de contexto para IA generativa em materiais educacionais abertos

Programação dinâmica e mochila 0/1 para seleção de chunks sob restrição de tokens: um estudo comparativo com estratégia gulosa

João Cosme Sena Sá · Eduardo Henrique do Lago Silva · Helena Carvalho Leal
Hernandison da Silva Bispo · Nadianne Maria dos Santos Galvão · Rafael Takeguma Goto

Tema	8 — Materiais educacionais abertos
Corpus	*Pense Python* (tradução livre da 3ª ed. de *Think Python*)
Repositório	`https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_N`
Licença do código	MIT · AV2 em desenvolvimento (tag pendente)
Vídeo da atividade	*Pendente de produção e publicação (AV2)*

Documento revisado em 29 de outubro de 2026 · São Cristóvão, SE
Entrega dos artefatos até 04/11/2026 · Apresentação em 05/11/2026

Estado desta versão do relatório.

Este documento apresenta a evolução do protótipo desenvolvido na AV1, incorporando a seleção de contexto sob restrição de tokens, conforme a Trilha A da Atividade 2.

A abordagem principal será a programação dinâmica para o problema da mochila 0/1, comparada a uma estratégia gulosa por relevância/token.

A especificação formal, a implementação, a análise de corretude, a complexidade e os resultados experimentais serão documentados nas respectivas seções, conforme o desenvolvimento da atividade.

Sumário

1 Identificação da equipe **2**

2 Tema, corpus e fonte **2**

2.1 Continuidade da Atividade 1 3

3 Repositório, licença e reprodutibilidade **3**

4 Especificação do problema **3**

4.1 Problema e representação dos candidatos 3

4.2 Relevância lexical 4

4.3 Função objetivo e restrições 4

4.4 Corretude e critérios de desempate 5

4.5 Estratégia gulosa e protocolo comparativo 5

5 Representação dos dados e estratégia de chunking **6**

5.1 Normalização e segmentação 6

5.2 Representação computacional 6

6 Algoritmos e pseudocódigo **6**

6.1 Programação dinâmica: mochila 0/1 6

6.2 Estratégia gulosa por relevância/custo 6

6.3 Justificativa de corretude 7

7 Análise no modelo RAM **7**

7.1 Programação dinâmica: mochila 0/1 7

7.2 Estratégia gulosa 7

7.3 Custo do pipeline completo 8

8 Melhor, pior e caso médio **8**

9 Recorrências **9**

9.1 Mochila 0/1 9

9.2 Merge Sort 10

10 Metodologia experimental **10**

10.1 Configuração experimental 10

10.2 Métricas e reprodutibilidade 11

11 Resultados e gráficos **11**

12 Discussão de escalabilidade **11**

13 Relação com IA generativa e RAG **11**

14 Limitações e ameaças à validade **11**

15 Declaração de uso de IA generativa **11**

16 Contribuição individual **12**

17 Vídeo da atividade **12**

18 Referências **12**

1. Identificação da equipe

A AV2 é desenvolvida por seis discentes do Programa de Pós-Graduação em Ciência da Computação (PROCC/UFS), na disciplina Projeto e Análise de Algoritmos, período 2026.2.

As responsabilidades abrangem especificação formal (Parte A), implementação e corretude (Parte B), análise assintótica (Parte C), experimentação (Parte D) e relação com IA generativa (Parte E).

A distribuição individual está registrada em `docs/CONTRIBUICOES.md` e resumida na Seção 16, com distinção entre responsabilidades acordadas e evidências de execução. A revisão de código, das provas e dos resultados é uma atividade coletiva.

2. Tema, corpus e fonte

O trabalho aborda o Tema 8 do enunciado: **materiais educacionais abertos**. O corpus é o livro *Pense Python*, tradução livre da terceira edição de *Think Python*, de Allen B. Downey, realizada por Rodrigo Castelan Carlson [7].

O recorte contém 21 notebooks Jupyter, incluindo prefácio, introdução e capítulos 1 a 19. A aquisição foi registrada em 02/09/2026, utilizando o commit `cfde2c450bf4f2ea65326da88e9968f400597e92`.

O material está disponível em `https://rodrigocarlson.github.io/PensePython3ed/`, com licença CC BY-NC-SA 4.0. O corpus não contém dados sensíveis.

2.1 Continuidade da Atividade 1

A AV2 preserva o corpus, a pontuação TF-IDF e a configuração de recuperação C3 da AV1, composta por busca linear e Merge Sort.

A nova etapa seleciona um subconjunto de chunks sob orçamento de unidades lexicais, modelado como mochila 0/1. A programação dinâmica constitui a abordagem principal, comparada a uma estratégia gulosa por relevância/custo [2].

São reutilizadas 30 consultas definidas na AV1, armazenadas em `data/queries.csv`, garantindo a comparação sobre o mesmo conjunto de perguntas.

O texto integral do corpus não é redistribuído no repositório. O script `scripts/baixar_corpus.py` permite reproduzir a aquisição no commit fixado, enquanto `data/corpus_manifest.csv` registra os hashes SHA-256 para verificação de integridade.

3. Repositório, licença e reprodutibilidade

O código-fonte está disponível no repositório público:

> `https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo`

O código utiliza licença MIT (`LICENSE`), distinta da licença CC BY-NC-SA 4.0 do corpus. A AV2 encontra-se em desenvolvimento na branch `av2-programacao-dinamica`; o commit ou a tag final será registrado na integração da entrega.

As dependências são documentadas em `requirements.lock`. O `README.md` apresenta as instruções de instalação, execução e reprodução dos experimentos. A versão do corpus é identificada pelo commit fixado e pelos hashes do manifesto, permitindo verificar a procedência dos dados utilizados.

4. Especificação do problema

4.1 Problema e representação dos candidatos

A AV2 investiga a seleção de um subconjunto de chunks recuperados pela configuração lexical C3 da AV1, buscando maximizar a relevância acumulada sob um orçamento de unidades lexicais. O problema é modelado como mochila 0/1, em que cada chunk é selecionado integralmente ou descartado [2].

O corpus corresponde ao livro *Pense Python*, tradução brasileira da terceira edição de *Think Python* [7]. O recorte da AV1 contém 21 notebooks Jupyter, segmentados em 2.737 chunks, dos quais 1.405 são de texto Markdown e 1.332 de código.

Cada candidato c_i possui identificador único, texto, custo lexical w_i, relevância v_i, campo `source_order` e metadados de origem. O custo é obtido de `n_tokens`, que representa unidades lexicais e não necessariamente tokens de um modelo de linguagem.

Para uma consulta q, os candidatos são definidos por:

$$C_q = \{c_1, c_2, \ldots, c_n\}, \tag{1}$$

em que $n = |C_q|$ é a quantidade de candidatos com relevância positiva, $w_i \in \mathbb{N}^+$ e $v_i \in \mathbb{R}_{\geq 0}$, com valores finitos. O orçamento máximo é $W \in \mathbb{N}_0$.

A entrada é representada por:

$$I = (q, C_q, W). \tag{2}$$

4.2 Relevância lexical

A pontuação dos candidatos preserva o mecanismo TF-IDF da AV1, fundamentado em frequência de termos e frequência inversa de documento [5].

Para uma consulta q e um chunk c, a relevância é calculada por:

$$\text{score}(q, c) = \sum_{t \in \text{TermosDistintos}(q)} qtf_t \, \text{tf}(t, c) \, \text{idf}(t), \tag{3}$$

em que qtf_t é a frequência do termo na consulta, $\text{tf}(t, c)$ sua frequência ponderada no chunk e $\text{idf}(t)$ sua frequência inversa documental.

A AV2 mantém a pontuação original, definindo:

$$v_i = \text{score}(q, c_i). \tag{4}$$

A implementação da Parte B converte esses escores em valores inteiros por $v_i^{(q)} = \text{round}(1000 v_i)$. Consequentemente, a otimalidade computacional refere-se aos valores quantizados, e não necessariamente à soma exata dos escores originais. Essa distinção é considerada na análise de corretude e nas recorrências.

A pontuação TF-IDF constitui uma estimativa lexical e não garante, isoladamente, cobertura semântica ou ausência de redundância.

4.3 Função objetivo e restrições

A saída é um subconjunto $S \subseteq C_q$, acompanhado do custo lexical total e da relevância acumulada:

$$W_S = \sum_{c_i \in S} w_i, \qquad V_S = \sum_{c_i \in S} v_i. \tag{5}$$

Define-se uma variável binária x_i, igual a 1 quando o candidato c_i é selecionado e 0 caso contrário.

A função objetivo da especificação é:

$$\max V_S = \max \sum_{i=1}^{n} v_i x_i. \tag{6}$$

Sujeita às restrições:

$$\sum_{i=1}^{n} w_i x_i \leq W, \tag{7}$$

$$x_i \in \{0, 1\}, \quad i = 1, \ldots, n. \tag{8}$$

O modelo impede a seleção fracionária e a repetição de candidatos. A restrição de capacidade utiliza a mesma unidade lexical para w_i e W.

4.4 Corretude e critérios de desempate

A corretude da seleção considera duas propriedades: viabilidade e otimalidade em relação à função objetivo adotada.

A viabilidade exige:

$$S \subseteq C_q \quad \text{e} \quad \sum_{c_i \in S} w_i \leq W. \tag{9}$$

Para a formulação original, uma solução ótima S^* satisfaz:

$$\sum_{c_i \in S^*} v_i \geq \sum_{c_i \in S'} v_i, \quad \forall S' \in \mathcal{F}, \tag{10}$$

em que $\mathcal{F}$ é o conjunto de subconjuntos viáveis.

Na implementação com programação dinâmica, a garantia de otimalidade se aplica à soma dos valores quantizados $v_i^{(q)}$. Entre soluções de igual valor quantizado, prioriza-se o menor custo total e, persistindo o empate, a menor sequência lexicográfica de `source_order`.

Casos como candidatos vazios, orçamento nulo, itens maiores que W, identificadores duplicados e relevância zero são definidos nos contratos da AV2 e verificados por testes.

4.5 Estratégia gulosa e protocolo comparativo

Como linha de base, utiliza-se uma estratégia gulosa que ordena os candidatos pela razão entre relevância e custo lexical:

$$r_i = \frac{v_i}{w_i}. \tag{11}$$

Na implementação, a razão utiliza os valores quantizados $v_i^{(q)}$. Os candidatos são examinados em ordem decrescente de densidade e selecionados quando cabem no orçamento restante. Empates são resolvidos por maior valor e, posteriormente, menor `source_order`.

Embora produza soluções viáveis, essa estratégia não garante otimalidade global para a mochila 0/1.

A comparação utilizará os mesmos candidatos recuperados pela configuração C3 da AV1, preservando corpus, consultas, custos, escores e orçamento. O planejamento prevê 30 consultas, $n \in \{10, 30, 50\}$ e $W \in \{256, 512, 1024\}$, com repetições definidas no protocolo experimental.

As métricas incluem relevância acumulada, uso do orçamento, tempo de seleção e consumo de memória. Os resultados experimentais serão apresentados na Parte D.

5. Representação dos dados e estratégia de chunking

A AV2 preserva o processamento do corpus da AV1, mantendo a normalização, a segmentação e a representação lexical dos chunks. A recuperação utiliza a configuração C3 (busca linear e Merge Sort), sem alterações nos critérios de pontuação e ordenação.

5.1 Normalização e segmentação

Nas células Markdown, aplica-se normalização NFKC, conversão para minúsculas, tratamento da pontuação e compactação de espaços, preservando os acentos. As células de código mantêm seu conteúdo e indentação originais, com representação lexical separada. Na configuração principal, as stopwords não são removidas e as saídas de execução são descartadas.

O chunking respeita a hierarquia de notebooks, capítulos e seções, sem combinar células Markdown e código. A configuração principal utiliza tamanho máximo de 256 unidades lexicais e sobreposição de 32, produzindo 2.737 chunks. As configurações 128/16 e 512/64 foram avaliadas na AV1, mas não são adotadas como padrão da AV2.

5.2 Representação computacional

Cada candidato é representado pelo par (w_i, v_i), acompanhado de identificador, `source_order` e metadados de origem. O valor v_i corresponde à relevância TF-IDF para a consulta, enquanto w_i é obtido do campo `n_tokens`.

O orçamento W limita a soma dos custos dos chunks selecionados. A contagem lexical utilizada como custo é uma aproximação, não necessariamente equivalente aos tokens de um modelo de linguagem.

A utilização de um único livro limita a generalização dos resultados. Além disso, a relevância lexical pode ser influenciada por termos técnicos em inglês, e a soma dos escores não modela diretamente redundância ou cobertura semântica.

6. Algoritmos e pseudocódigo

A AV2 reutiliza o mecanismo de recuperação lexical da AV1, adotando a configuração C3, composta por busca linear e Merge Sort.

A principal alteração consiste na inclusão de uma etapa de seleção de contexto sob restrição de unidades lexicais, modelada como mochila 0/1.

Serão consideradas duas abordagens: programação dinâmica, como algoritmo principal, e uma estratégia gulosa por relevância/custo, utilizada como linha de base.

6.1 Programação dinâmica: mochila 0/1

6.2 Estratégia gulosa por relevância/custo

6.3 Justificativa de corretude

7. Análise no modelo RAM

A análise considera a implementação da Parte B, composta pela programação dinâmica para a mochila 0/1 e pela estratégia gulosa por relevância/custo. Adota-se o modelo RAM, no qual operações elementares sobre valores de tamanho limitado possuem custo constante. Para estruturas hash, considera-se custo esperado constante.

Sejam n a quantidade de candidatos, W o orçamento em unidades lexicais e k o número de chunks selecionados, com $0 \leq k \leq n$. Na recuperação C3 da AV1, N representa a quantidade de chunks do corpus, P a quantidade de resultados positivos, Q o comprimento da consulta e L a soma dos comprimentos dos textos do corpus.

7.1 Programação dinâmica: mochila 0/1

A função `mochila_dp()` valida os candidatos, ordena-os por `source_order` e constrói uma matriz E com $(n+1)(W+1)$ posições.

O preenchimento percorre exatamente $n(W+1)$ estados. Posteriormente, a solução é reconstruída mediante um percurso dos n candidatos, seguido da ordenação dos k chunks selecionados.

A validação possui custo esperado $O(n)$, enquanto as ordenações realizadas por `sorted()`, baseado em Timsort, possuem limite superior $O(n \log n)$.

Assim, para $n \geq 1$ e $W \geq 1$:

$$T_{\mathrm{DP}}(n, W) = \Theta(nW) + O(n \log n). \tag{12}$$

No pior caso, o custo total é $\Theta(nW + n \log n)$. Quando $W = \Omega(\log n)$, o preenchimento da matriz domina ou iguala assintoticamente as ordenações, resultando em $\Theta(nW)$.

Como a implementação mantém a tabela completa para reconstruir a solução e aplicar os desempates, sua complexidade espacial é:

$$M_{\mathrm{DP}}(n, W) = \Theta((n+1)(W+1)). \tag{13}$$

Caso fosse necessário calcular apenas o valor ótimo, uma implementação com duas linhas poderia utilizar $O(W)$ de memória. Essa otimização não corresponde ao código avaliado.

A complexidade é pseudopolinomial, pois depende do valor numérico de W, e não apenas de sua representação em bits.

7.2 Estratégia gulosa

A função `guloso_densidade()` ordena os candidatos pela razão entre valor quantizado e custo lexical. Em seguida, percorre os n candidatos e seleciona integralmente aqueles que cabem no orçamento restante.

A validação e a seleção apresentam custo linear. A ordenação inicial e a ordenação final dos k escolhidos possuem limites superiores $O(n \log n)$ e $O(k \log k)$, respectivamente.

Como $k \leq n$, obtém-se:

$$T_{\mathrm{G}}(n) = O(n \log n). \tag{14}$$

O espaço auxiliar é:

$$M_{\mathrm{G}}(n) = \Theta(n). \tag{15}$$

Diferentemente da programação dinâmica, o custo assintótico do guloso não depende diretamente do valor numérico de W. Entretanto, a estratégia não garante otimalidade para a mochila 0/1.

7.3 Custo do pipeline completo

A configuração C3 da AV1 realiza busca linear, cálculo lexical TF-IDF e ordenação dos P resultados positivos por Merge Sort.

Considerando a normalização das consultas e dos textos em cada recuperação, o custo da busca é $\Theta(NQ + L)$. O Merge Sort apresenta $\Theta(1 + P \log(P + 1))$.

Após a recuperação, a conversão dos n resultados em objetos `Item`, por `itens_de_resultados()`, possui custo esperado $\Theta(n)$.

Consequentemente, os limites superiores dos pipelines são:

$$\begin{aligned} T_{\text{total,DP}} = O(&NQ + L + P \log(P + 1) \\ &+ nW + n \log n), \end{aligned} \tag{16}$$

$$\begin{aligned} T_{\text{total,G}} = O(&NQ + L + P \log(P + 1) \\ &+ n \log n). \end{aligned} \tag{17}$$

O custo linear de conversão é absorvido pelos demais termos. A diferença entre os pipelines decorre principalmente da estratégia de seleção.

As expressões não incluem, por consulta, a construção inicial das estatísticas documentais, realizada uma vez na preparação do corpus.

8. Melhor, pior e caso médio

Programação dinâmica. O preenchimento da matriz executa exatamente $n(W + 1)$ iterações, independentemente dos valores e custos dos candidatos ou da solução selecionada.

Assim, a inicialização e o preenchimento possuem custo $\Theta((n + 1)(W + 1))$ em todos os casos de entrada válida.

O custo adicional das ordenações depende da disposição dos elementos. Sob o modelo RAM adotado, os limites assintóticos são:

$$\begin{aligned} T_{\mathrm{DP}}^{\mathrm{melhor}}(n, W) &= \Theta(nW), \\ T_{\mathrm{DP}}^{\mathrm{pior}}(n, W) &= \Theta(nW + n \log n). \end{aligned} \tag{18}$$

Estratégia gulosa. O percurso de seleção examina todos os n candidatos. Entretanto, as ordenações podem apresentar comportamentos diferentes conforme a disposição das chaves.

Como o Timsort pode explorar sequências favoráveis à ordenação, os limites são:

$$\begin{aligned} T_{\mathrm{G}}^{\mathrm{melhor}}(n) &= \Theta(n), \\ T_{\mathrm{G}}^{\mathrm{pior}}(n) &= \Theta(n \log n). \end{aligned} \tag{19}$$

O melhor caso linear não deve ser presumido para as consultas reais, pois a recuperação C3 ordena os candidatos por relevância, enquanto o guloso os ordena por densidade de relevância/custo.

Caso médio. Uma análise probabilística rigorosa exige definir uma distribuição de entradas. Como essa distribuição não foi estabelecida para o corpus e as consultas avaliados, não se atribui uma expressão específica ao caso médio.

Os limites apresentados constituem garantias assintóticas, enquanto os tempos observados serão investigados na Parte D.

9. Recorrências

9.1 Mochila 0/1

Na implementação da Parte B, o valor computacional de cada item é quantizado pela escala 1000:

$$v_i^{(q)} = \mathrm{round}(1000 v_i),$$

em que v_i é o escore TF-IDF original e `round()` segue o arredondamento do Python.

A chave utilizada pela programação dinâmica é:

$$a_i = v_i^{(q)}(W+1) - w_i. \tag{20}$$

Essa representação prioriza a maior soma dos valores quantizados e, em caso de igualdade, o menor custo. O desempate por sequência lexicográfica de `source_order` ocorre durante a reconstrução.

Seja $E(i, w)$ a maior chave obtida a partir dos candidatos de i até $n-1$, com capacidade w.

A condição inicial é $E(n, w) = 0$. A recorrência é:

$$E(i,w) = \begin{cases} E(i+1, w), & w_i > w, \\ \max\{E(i+1,w),\ a_i + E(i+1, w-w_i)\}, & w_i \le w. \end{cases} \tag{21}$$

A implementação calcula cada estado uma única vez. Sua recorrência temporal de preenchimento é:

$$\begin{aligned} C(i, W) &= C(i-1, W) + \Theta(W+1), \\ C(0, W) &= \Theta(W+1). \end{aligned} \tag{22}$$

Por expansão:

$$C(n, W) = \Theta((n+1)(W+1)). \tag{23}$$

A otimalidade da implementação refere-se aos valores quantizados, não necessariamente à soma exata dos escores originais.

9.2 Merge Sort

Na configuração C3, o Merge Sort divide os P resultados positivos em duas partes aproximadamente iguais.

Para $P \geq 2$, sua recorrência é:

$$T(P) = T(\lfloor P/2 \rfloor) + T(\lceil P/2 \rceil) + \Theta(P). \tag{24}$$

Com $T(0) = T(1) = \Theta(1)$.

Para P potência de dois:

$$T(P) = 2T(P/2) + \Theta(P).$$

Pelo Teorema Mestre, $a = 2$, $b = 2$ e $f(P) = \Theta(P)$.

Como $P^{\log_b a} = P$, aplica-se o segundo caso:

$$T(P) = \Theta(P \log P), \quad P \geq 2. \tag{25}$$

O resultado também se aplica a tamanhos que não são potências de dois: a recorrência com piso e teto produz profundidade $\Theta(\log P)$ e custo total $\Theta(P)$ por nível, até fatores constantes.

Incluindo as entradas vazias e unitárias, pode-se escrever:

$$T(P) = \Theta(1 + P \log(P+1)).$$

A fundamentação detalhada das recorrências e as verificações computacionais são apresentadas nos apêndices técnicos.

10. Metodologia experimental

10.1 Configuração experimental

A avaliação compara a programação dinâmica para mochila 0/1 com a estratégia gulosa por relevância/custo, utilizando os mesmos candidatos recuperados pela configuração C3 da AV1 (busca linear e Merge Sort).

O planejamento contempla 30 consultas, armazenadas em `data/queries.csv`, com quantidades de candidatos $n \in \{10, 30, 50\}$ e orçamentos $W \in \{256, 512, 1024\}$ unidades lexicais. São considerados apenas os candidatos disponíveis com relevância positiva.

Cada cenário prevê pelo menos três repetições, sujeitas à viabilidade experimental. Ambas as estratégias recebem os mesmos custos, escores e limites de orçamento.

10.2 Métricas e reprodutibilidade

As métricas de qualidade incluem relevância acumulada (V_S), custo lexical utilizado (W_S), taxa de utilização do orçamento (W_S/W, para $W > 0$) e redundância entre os chunks selecionados.

As métricas computacionais incluem tempo de seleção, tempo total do pipeline e consumo aproximado de memória.

Para permitir a reprodução dos experimentos, serão registrados os parâmetros de execução, a versão do corpus, a configuração C3, o ambiente computacional, as dependências e o commit correspondente à AV2.

O mecanismo de recuperação permanecerá fixo nas comparações, de modo que as diferenças observadas sejam atribuídas às estratégias de seleção.

11. Resultados e gráficos

12. Discussão de escalabilidade

13. Relação com IA generativa e RAG

A AV2 acrescenta uma etapa de seleção de contexto a um fluxo de geração aumentada por recuperação (*Retrieval-Augmented Generation* – RAG) [6].

Após a recuperação lexical C3 da AV1, a mochila 0/1 seleciona chunks sob um orçamento de unidades lexicais, buscando maximizar a relevância acumulada. A programação dinâmica é comparada à estratégia gulosa por relevância/custo.

A solução ótima para a função objetivo adotada não implica necessariamente respostas melhores de um LLM, pois não considera diretamente diversidade, redundância ou cobertura semântica.

14. Limitações e ameaças à validade

As principais limitações são: (i) a relevância TF-IDF depende de correspondência lexical; (ii) a função objetivo aditiva não penaliza redundância entre chunks; (iii) o campo `n_tokens` representa unidades lexicais, não tokens exatos de um LLM; e (iv) a utilização de um único livro limita a generalização.

Além disso, a seleção depende dos candidatos recuperados pela configuração C3. Chunks relevantes não recuperados não poderão ser selecionados posteriormente.

A validade dos resultados também depende das consultas, orçamentos, quantidades de candidatos e condições computacionais avaliadas.

15. Declaração de uso de IA generativa

Ferramentas de IA generativa foram utilizadas como apoio à revisão textual e à estruturação metodológica da especificação.

O registro de ferramentas, modelos, finalidades, prompts e sugestões aceitas ou rejeitadas é mantido em `docs/USO_DE_IA.md`.

A equipe permanece responsável pela revisão das formulações matemáticas, validação do código, demonstrações de corretude e interpretação dos resultados. Sugestões geradas por IA não constituem, isoladamente, evidências científicas.

16. Contribuição individual

As responsabilidades individuais foram distribuídas da seguinte forma: João (Partes B e D); Eduardo (B, C e D); Helena (D e E); Hernandison (A e E); Nadianne (A e C); Rafael (B e E).

As evidências de execução, incluindo commits, arquivos, revisões e resultados, são documentadas em `docs/CONTRIBUICOES.md`. A avaliação crítica e a revisão do trabalho são responsabilidades coletivas.

17. Vídeo da atividade

O vídeo de apresentação da AV2 documentará a implementação, as verificações de corretude e os resultados experimentais.

A URL será registrada neste relatório, no `README.md` e no `VIDEO.md`, após sua publicação.

18. Referências

Referências

[1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.

[2] KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Pearson/Addison-Wesley, 2006.

[3] SKIENA, S. S. *The Algorithm Design Manual*. 3. ed. Springer, 2020.

[4] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.

[5] MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[6] LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 2020. Disponível em: `https://arxiv.org/abs/2005.11401`.

[7] DOWNEY, A. B. *Think Python*. 3. ed. O'Reilly Media, 2024. Tradução brasileira: *Pense Python*, por Rodrigo Castelan Carlson. Disponível em: `https://rodrigocarlson.github.io/PensePython3ed/`.

[8] PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.